



Debreceni Egyetem, Informatikai Kar

IT Biztonság

OpenSSL

Titkosítás DES és AES algoritmusokkal

Titkosítás DES algoritmussal

Előkészületek

A következőkbe az OpenSSL programcsomag három olyan parancsát fogjuk megvizsgálni, melyek DES algoritmus segítségével titkosítanak: `des-ecb`, `des-cbc` és `des-ede-cbc`. Néhány fontos opció:

- `-d` opció jelzi, ha titkosítás helyett visszafejtésre használjuk a parancsot,
- `-in` opció után definiálható a bementi állomány,
- `-out` opció után definiálható a kimeneti állomány,
- `-K` opció után adjuk meg a titkosításra használt kulcsot hexadecimális formában,
- `-p` opció használatával kérhetünk információt a titkosításhoz használt paraméterekről.

Készítsünk két állományt melyet az algoritmuok teszteléséhez használunk. Egy `date` nevűt, ami az aktuális dátumot tartalmazza:

```
$ date >date
$ cat date
Thu Apr 23 19:25:26 CEST 2020
```

és egy `zeros` nevűt, amiben 6 darab 8 bájt méretű nullákat tartalmazó blokk (48 bájt) van:

```
$ dd if=/dev/zero of=zeros bs=8 count=6
6+0 records in
6+0 records out
48 bytes (48 B) copied, 0.000587394 s, 81.7 kB/s
```

Ezen állományok tartalmát hexadecimális alakban is megtekinthetjük (a `vim` csomagban található) `xxd` program segítségével.

```
$ xxd date
00000000: 5468 7520 4170 7220 3233 2032 303a 3238  Thu Apr 23 20:28
00000010: 3a33 3120 4345 5354 2032 3032 300a      :31 CEST 2020.
```

```
$ xxd zeros
00000000: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  .....
```

Titkosítás DES algoritmussal

Electronic codebook (ECB)

A DES algoritmus használatának legegyszerűbb esete, amikor a titkosítandó információt 8 bájtos blokkokra darabolva blokkonként (külön-külön) dolgozzuk fel. Ehhez választanunk kell egy szintén 8 bájtos méretű kulcsot. Például: AC22B54854967CA9. Titkosítsuk a date állományt.

```
$ openssl des-ecb -in date -K AC22B54854967CA9 -out date.enc
```

A szimmetrikus kulcsú kriptográfia esetén a visszafejtéshez ugyanezen kulcs használandó:

```
$ openssl des-ecb -d -in date.enc -K AC22B54854967CA9
```

```
Thu Apr 23 19:25:26 CEST 2020
```

Az ECB hátránya, hogy azonos tartalmú blokkok ugyanazon kulccsal történő titkosításának eredménye megegyezik. Jól látszik ez, ha a zeros állományon végezzük el a műveletet:

```
$ openssl des-ecb -in zeros -K AC22B54854967CA9 -nosalt | xxd
```

```
00000000: f695 a6a8 5eba 1595 f695 a6a8 5eba 1595  ....^.....^...
00000100: f695 a6a8 5eba 1595 f695 a6a8 5eba 1595  ....^.....^...
00000200: f695 a6a8 5eba 1595 f695 a6a8 5eba 1595  ....^.....^...
00000300: 7297 73cc e846 c06c                      r.s..F.l
```

Mivel az eredeti állomány mind a 6 nyolc bájtos blokkja azonos volt, így a titkosított szöveg első 6 nyolc bájtos blokkja is megegyezik. Az utolsó blokk a padding-nak nevezett rész. (A padding feladata, hogy blokkmérettel osztható méretűvé egészítse ki a titkosítandó szöveget, e mellett pedig tartalma tárolja az eredeti bemenet hosszát, hogy az pontosan visszaállítható legyen.)

Az ECB – amint azt a fenti példa is mutatja – nem biztonságos, mert esetenként a titkosított tartalomból következtetni lehet az eredeti szöveg tulajdonságaira.

Cipher block chaining (CBC)

A CBC eljárás során is blokkonként titkosítunk, de az előző blokkból képzett eredmény a következő blokk kódolása előtt xor művelettel hozzáadódik a blokkhoz. Ennek eredménye az, hogy két azonos tartalmú egymást követő blokk más-más kimenetet eredményez. A kérdés csupán az, hogy mit adjunk hozzá az első blokkhoz. Ezt a bitsorozatot hívjuk inicializáló vektornak és -iv kapcsoló után adjuk meg a CBC algoritmusok esetén. (Nem feltétlenül a 0 a legjobb választás, véletlenszerűen szokás generálni.)

```
$ openssl des-cbc -in zeros -K AC22B54854967CA9 -iv 0 -nosalt | xxd
```

```
00000000: f695 a6a8 5eba 1595 4260 6a3f 9420 ba2f  ....^...B'j?. ./
00000100: 5fe6 c40b 8f88 6564 a195 d672 ba2e 6020  _.....ed...r..'
00000200: ffb4 1708 c446 e753 eb98 8a2f 9ed4 ffe8  ....F.S.../....
00000300: 39dc 5883 aca3 e4b2                      9.X.....
```

Annak, aki vissza szeretné fejteni az üzenetet, a kulcsot is és az inicializáló vektort is ismernie kell. Az inicializáló vektort hiányában csak az első 8 bájtos blokk visszafejtése sikertelen, így az is megoldás lehet, ha az első 8 bájtban nem tárolunk hasznos információt.

Titkosítás DES algoritmussal

Kulcsgenerálás jelszó segítségével

A megfelelő kulcsok előállítására jelszavakat is használhatunk. A jelszó és egy véletlen bájtorozat (salt) segítségével állítható elő a megfelelő kulcs. A véletlen bájtorozat biztosítja, hogy ugyanazon jelszó használata esetén is különböző kulcsok keletkezzenek. A véletlen bájtorozat (salt) a titkosított szövegben eltárolásra kerül. Aki a jelszót ismeri, ennek segítségével elő tudja állítani a visszafejtéshez szükséges kulcsot. Kapcsolódó opciók:

- -k után adható meg a jelszó (ha ez elmarad, akkor bekéri majd az openssl),
- -S a kulcsgeneráláshoz szükséges véletlen bájtorozat (salt) hexadecimális formában.

A következő példában a -S opció elmara, így véletlenszerűen generálja az openssl. Ezt (ahogy a most szintén véletlenszerűen generált inicializáló vektort is) a -p opció segítségével kérdezzük le.

```
$ openssl des-cbc -k PASSWORD -in date -out date.1 -p
salt=4C8B4816F733C20E
key=5E24A748C17A8BA3
iv =489C75F095D413D1
```

Ha ismerjük a kulcsot és az inicializáló vektort is, akkor könnyen reprodukálható az eredmény.

```
$ openssl des-cbc -K 5E24A748C17A8BA3 -iv 489C75F095D413D1 -in date -out date.2 -p
salt=3C000000481009B7
key=5E24A748C17A8BA3
iv =489C75F095D413D1
```

A salt ebben az esetben is legenerálásra kerül, de ez felesleges. A -nosalt opcióval jelezhetnénk is, hogy nem kell generálni. A fenti két parancs eredménye mindössze annyiban különbözik, hogy amennyiben a kulcsot jelszó és salt segítségével generáljuk, úgy a salt értéke eltárolásra kerül az eredményben:

```
$ xxd date.1
00000000: 5361 6c74 6564 5f5f 4c8b 4816 f733 c20e  Salted__L.H..3..
00000100: 05bf 507c b151 679e db20 152c a99b 64ef  ..P|.Qg.. ,...d.
00000200: d10f ba50 3c47 6972 225d 8708 592c 0c41  ...P<Gir"]..Y,.A
```

```
$ xxd date.2
00000000: 05bf 507c b151 679e db20 152c a99b 64ef  ..P|.Qg.. ,...d.
00000100: d10f ba50 3c47 6972 225d 8708 592c 0c41  ...P<Gir"]..Y,.A
```

Mindkét esetben a visszafejtéshez csak a -d opció hozzáadása szükséges:

```
$ openssl des-cbc -d -k PASSWORD -in date.1
Thu Apr 23 19:25:26 CEST 2020
```

A jelszóból és a salt-ból az inicializáló vektor és a kulcs is előállítható volt. Ellenben:

```
$ openssl des-cbc -d -K 5E24A748C17A8BA3 -iv 489C75F095D413D1 -in date.2
Thu Apr 23 19:25:26 CEST 2020
```

TDES

A TDES (triple DES) algoritmus úgy javítja a DES hatékonyságát, hogy 1 helyett 3 átalakítást végez: titkosít, visszafejt, majd ismét titkosít egy blokkot (encrypt-decrypt-encrypt). A három átalakításhoz használt kulcsok akár egybe is eshetnek, amely esetben a DES működését kapjuk vissza. Általában az első és a harmadik kulcs ugyanaz, a második különböző (akár ugyanazon jelszóból más véletlen salt segítségével előállítva).

```
$ openssl des-ede-cbc -in date -k PASSWORD -out date.tdes -p  
salt=8BA201F9E5D9AA6E  
key=12D5B0358F319E8ED06728767B65C919  
iv =EF7E2BA510D11284
```

Figyeljük meg, hogy a kulcs hossza duplájára nőtt (valójában két kulcsunk van). A visszafejtés megint csak a -d kapcsolóban (és a ki- és bemenetben) különbözik:

```
$ openssl des-ede-cbc -d -in date.tdes -k PASSWORD  
Thu Apr 23 19:25:26 CEST 2020
```

Titkosítás AES algoritmussal

Az AES algoritmus esetében hasonlóan alakulna a parancsok.

```
$ openssl aes-128-cbc -k PASSWORD -in date -out date.aes -p
```

```
salt=64B963E76B7F0FAE
```

```
key=E5B9978C012AE1B7AE0DB1964BFD0A94
```

```
iv =068D753899FB696348B2FA6B593D3E80
```

```
$ openssl aes-128-cbc -k PASSWORD -d -in date.aes
```

```
Thu Apr 23 19:25:26 CEST 2020
```